

UNIT 3: ADVANCED INTERNAL SEARCH METHODS

Residue-Based, Digital Data Structures, Spatial Organization, Indexing

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-II



Course Outline

- 1 Residue-based Searches
- 2 Digital Data Structures
- 3 Spatial Organization Methods
- 4 In-Memory Indexing & Analysis



Outline

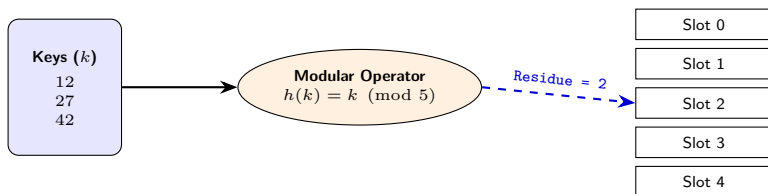
- 1 Residue-based Searches
- 2 Digital Data Structures
- 3 Spatial Organization Methods
- 4 In-Memory Indexing & Analysis



Residue-Based Search Models I

Residue Search

An advanced retrieval framework using mathematical properties of **modular arithmetic equivalences** to map partitions of keys into distinct address spaces.



* Note that:

$12 \equiv 27 \equiv 42 \pmod{5}$
They form an equivalence class.



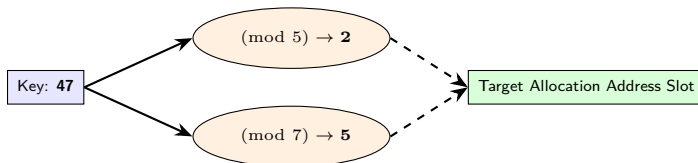
Residue-Based Search Models II

Multiple Residue Search

Improves simple **key-to-slot mapping** distributions by running the identifier through a cascade of distinct modular operations simultaneously.

$$\mathbf{H}(\mathbf{k}) = \{\mathbf{k} \pmod{\mathbf{m}_1}, \mathbf{k} \pmod{\mathbf{m}_2}\}$$

This **multi-residue intersection pattern** drastically reduces clustered placement conflicts.



Outline

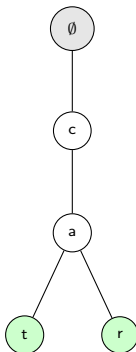
- 1 Residue-based Searches
- 2 Digital Data Structures**
- 3 Spatial Organization Methods
- 4 In-Memory Indexing & Analysis



Digital Search Trees: Tries

Definition (Trie / Prefix Tree)

An **ordered tree data structure** where nodes *do not* store their actual search keys. Instead, a node's **position path in the tree hierarchy** dictates the associated *string index key value*.



Key Benefits:

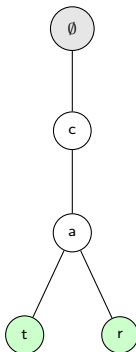
- Search runtime is tied strictly to **key length (L)**, completely independent of dataset size (n).
- Complexity: $O(L)$.
- Ideal for auto-complete dictionaries and IP routing path evaluations.



Digital Search Trees: Tries

Definition (Trie / Prefix Tree)

An **ordered tree data structure** where nodes *do not* store their actual search keys. Instead, a node's **position path in the tree hierarchy** dictates the associated *string index key value*.



Key Benefits:

- Search runtime is tied strictly to **key length (L)**, completely independent of dataset size (n).
- Complexity: **$O(L)$** .
- Ideal for auto-complete dictionaries and IP routing path evaluations.



Outline

- 1 Residue-based Searches
- 2 Digital Data Structures
- 3 Spatial Organization Methods**
- 4 In-Memory Indexing & Analysis



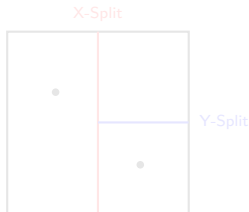
Spatial Search Methods

The Grid Method

Divides a *continuous multi-dimensional search coordinate space* into **uniform rectangular geometric cells (bins)**. Eliminates checking unrelated distant spaces.

2D Trees (k-d Trees)

A space-partitioning data structure that alternates geometric axis coordinate boundaries (**X-axis split**, then **Y-axis split**) at each successive tree tier depth.



Application Use Cases: Range lookups, nearest neighbor geographic calculations, and multi-variable collision handling patterns.



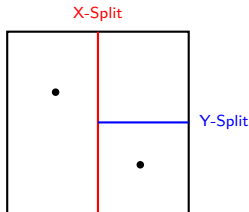
Spatial Search Methods

The Grid Method

Divides a *continuous multi-dimensional search coordinate space* into **uniform rectangular geometric cells (bins)**. Eliminates checking unrelated distant spaces.

2D Trees (k-d Trees)

A space-partitioning data structure that alternates geometric axis coordinate boundaries (**X-axis split**, then **Y-axis split**) at each successive tree tier depth.



Application Use Cases: Range lookups, nearest neighbor geographic calculations, and multi-variable collision handling patterns.



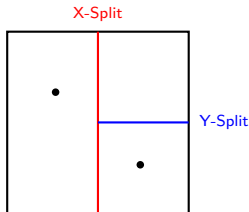
Spatial Search Methods

The Grid Method

Divides a *continuous multi-dimensional search coordinate space* into **uniform rectangular geometric cells (bins)**. Eliminates checking unrelated distant spaces.

2D Trees (k-d Trees)

A space-partitioning data structure that alternates geometric axis coordinate boundaries (**X-axis split**, then **Y-axis split**) at each successive tree tier depth.



Application Use Cases: Range lookups, nearest neighbor geographic calculations, and multi-variable collision handling patterns.



Outline

- 1 Residue-based Searches
- 2 Digital Data Structures
- 3 Spatial Organization Methods
- 4 In-Memory Indexing & Analysis**



In-Memory Index Tables

Index Table

A lightweight, **auxiliary structure** containing *ordered key fields* alongside corresponding *memory address references*. Speeds up access without reorganizing massive raw underlying records.



Performance Metrics

Advanced Methods Efficiency Matrices

Data Structure	Average Search	Space Overhead	Dimensionality
Multiple Residue	$O(1)$	Medium (Sparse)	1D Unique Keys
Digital Tree (Trie)	$O(L)$	High (Pointers)	Sequential Strings
Grid Structure	$O(1)$ Avg	High Array Scale	Multi-D Coordinates
k-d Spatial Tree	$O(\log n)$	Linear $O(n)$	k -Dimensions

Algorithmic Selection Rule

Choose **Tries** when working with string collections sharing prefix sub-patterns. Choose **k-d Trees** when dealing with spatial bounds or dynamic geometric coordinates.



Performance Metrics

Advanced Methods Efficiency Matrices

Data Structure	Average Search	Space Overhead	Dimensionality
Multiple Residue	$O(1)$	Medium (Sparse)	1D Unique Keys
Digital Tree (Trie)	$O(L)$	High (Pointers)	Sequential Strings
Grid Structure	$O(1)$ Avg	High Array Scale	Multi-D Coordinates
k-d Spatial Tree	$O(\log n)$	Linear $O(n)$	k -Dimensions

Algorithmic Selection Rule

Choose **Tries** when working with string collections sharing prefix sub-patterns. Choose **k-d Trees** when dealing with spatial bounds or dynamic geometric coordinates.



Outline

- 1 Residue-based Searches
- 2 Digital Data Structures
- 3 Spatial Organization Methods
- 4 In-Memory Indexing & Analysis



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-ii>

